

ECE 315

Steven Knudsen, PhD, PEng

Lecture 7 :

- Bare Metal Programming
- Strategies to ensure deterministic, low-latency performance
- More on RTOS scheduling



**UNIVERSITY
OF ALBERTA**

Last Lecture

- Embedded Software
- Design Thoughts
- Real-Time Specifications and Constraints

This Lecture

- Bare Metal Programming
- Strategies to ensure deterministic, low-latency performance

Learning Objectives

- Recognize when bare metal programming is appropriate
- Name and understand the advantages and drawbacks of bare metal
- Appreciate the need for excess computational capacity
- Appreciate the need to write efficient and deterministic interrupt service routines
- Appreciate the hazards of interrupt-driven architectures

“Bare Metal”

- Program without operating system
- It interacts directly with the processor, memory, and peripherals through registers and low-level hardware interfaces

“Bare Metal”

- Program without operating system
- It interacts directly with the processor, memory, and peripherals through registers and low-level hardware interfaces

- **Benefits include**

- Simplicity; minimum of support code
- Highly deterministic operation; timing is “tight”
- Program execution is the most efficient possible
- Best use of scarce resources

- **Drawbacks include**

- No access to standard and convenient utilities, functions found in an OS
- More complex development
- Poor portability – tied to processor
- Challenging to debug
- Greater learning curve

Single-threaded Architectures

- Typically, bare metal
- Foreground-background systems
- Sleep mode with interrupts
- One non-preemptive loop (no interrupts), or
- One non-preemptive loop with interrupts
- Multiple non-preemptive loops with interrupts
- ...others...

Foreground-Background Systems

- Simplest is one thread executing in a single non-preemptive loop
 - In C, just `main( )`
- Can manage soft- or non-real-time needs → **background processing**
- Latency and variability of response can be high, maybe too high...
- Interrupt service routines (ISRs) provide fast response to critical events → **foreground processing**
- Breaks out of main loop to do time-critical processing

- These systems are common in applications that are “human speed”

Sleep Mode with Interrupts

- Especially for power-sensitive applications, keeping the processing in sleep state as much as possible is a benefit
- Usually this precludes real-time response to events, but modern architecture has “smart peripherals” on the processor chip that can execute simple state machines and, if necessary, wake the processor up

Zynq and Low-power Operation

Sleep modes:

- Active
 - Full operation; high power consumption
- Standby
 - CPUs halted; some peripherals off
 - Memory retained; Quick wakeup
- Sleep
 - CPUs halted; most peripherals off
 - Memory retained; Moderate wakeup
- Deep Sleep
 - Just about all off, Very low power
 - Memory lost; Long wakeup

Wakeup actions:

- Programmable Logic
 - May remain active in all modes
 - Can itself wake up the processor(s)
- Wakup sources
 - Interrupts
 - Timer
 - Real-time clock (RTC)
 - External signals (interrupts)
 - Power management unit (PMU); not universal
- Wakeup behaviour
 - Standby/Sleep: Wakeup resumes operation almost immediately at the same execution point.
 - Deep Sleep: System may require reinitialization steps depending on how much was powered off.

RP2040 and Low-power Operation

Sleep modes:

- Active
 - Full operation; high power consumption
- Sleep
 - One or both CPUs halted
 - All peripherals on
 - Memory retained; Quick wakeup
- Dormant
 - CPUs halted; almost all peripherals off
 - Memory retained; μ seconds to wakeup
- Shutdown
 - Power to chip cutoff
 - Memory lost; slow to wake up



Wakeup actions:

- From Sleep
 - Any interrupt, internal or external
- From Dormant
 - Only sources specified in DORMANT_WAKE register can wake
 - GPIO edge (rising/falling as configured)
 - Real-Time Clock (RTC) alarm
 - XOSC/ROSC start-up event
- From Shutdown
 - External power controller cycles power

Made with help from ChatGPT

11

One Non-preemptive Loop (no interrupts)

- Aka ***static non-preemptive scheduling***
- Input sources/devices polled, possibly at different rates, in single loop
- The longest possible execution time of the routines possible inside the loop must be shorter than the loop period
 - Loop period dictates the overall worst case latency
 - Variability of the inner loop routine execution times, standalone and combined, dictates the overall variability
- Loop timing best defined by a hardware timer (not software)
- A final waiting sub-loop at the end of the loop uses up the remaining time each iteration

One Non-preemptive Loop (no interrupts)

```
initialization_steps();  
static int flag = 0;  
while (1) {  
    /* wait for the timer ISR to set flag to 1 */  
    while ( !flag ) {} /* wait in idling loop */  
    flag = 0; /* cleared for next iteration */  
    /* Start executing the nonpreemptive loop */  
    poll_A(); /* service first device */  
    poll_B(); /* service second device */  
    poll_C(); /* service third device */  
} /* end while */
```



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

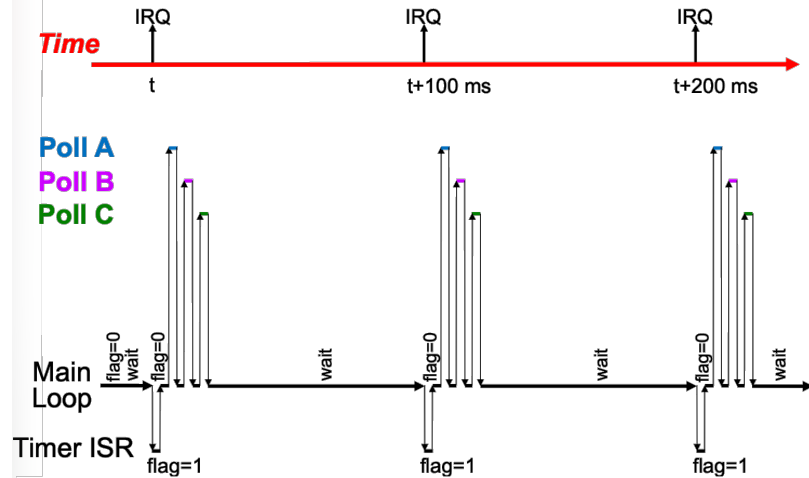
13

Why is there a timer?

Why not a sleep() statement in the loop?

One Non-preemptive Loop (no interrupts)

- Advantages
 - Simplicity
 - Guaranteed min polling freq
 - Guaranteed max response time
- Disadvantages
 - Poor determinism
 - Slow response
 - May poll too fast



One Non-preemptive Loop (interrupts)

- Add a few interrupts to previous scheme
 - Ensures physical signal-triggered events are serviced deterministically
- Each interrupt service routine (ISR) must be short
 - If longer processing is needed, have ISR set things up, then do in main loop
 - E.g., set a flag to indicate that more processing is needed
- Keep the processor lightly or moderately loaded so ISRs don't impact too much the main loop timing

One Non-preemptive Loop (interrupts)

```
initialization_steps();  
static int flag = 0;  
while (1) {  
    /* wait for the timer ISR to set flag to 1 */  
    while ( !flag ) {} /* wait in idling loop */  
    flag = 0; /* cleared for next iteration */  
    /* Start executing the nonpreemptive loop */  
    poll_A(); /* service first device */  
    poll_B(); /* service second device */  
    poll_C(); /* service third device */  
    process_data_inputs_from_other_ISR();  
    transfer_data_outputs_to_other_ISR();  
} /* end while */
```

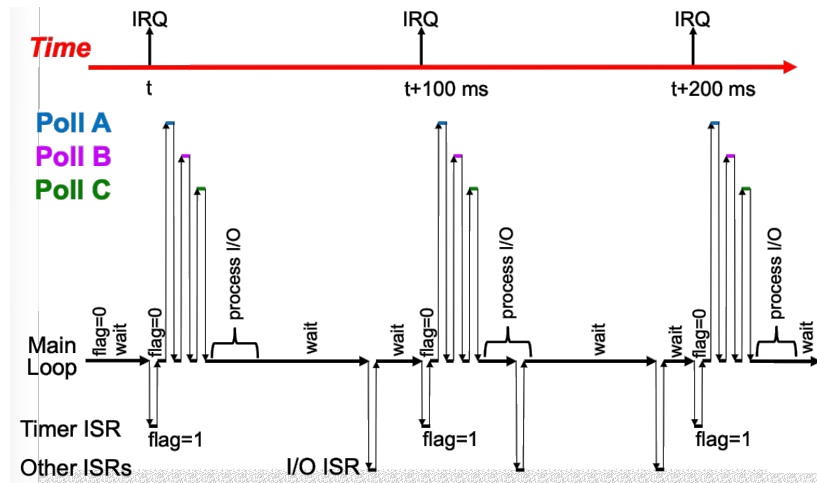


Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

16

One Non-preemptive Loop (interrupts)

- Advantages
 - Simplicity
 - Guaranteed min polling freq
 - Predictable max response time
- Disadvantages
 - Poor determinism
 - Slow response
 - May poll too fast



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

17

Multiple non-preemptive loops with interrupts

- Create two or more loops so that tempos of operation can be different
 - E.g., gather all short time events into one loop, longer time events in another
- Think of it as tailoring for event durations
- Must still be single thread – *loops run inside one outer loop*
- Otherwise very similar to single non-preemptive loop
 - Each interrupt service routine (ISR) must be short
 - Keep the processor lightly or moderately loaded so ISRs don't impact too much the main loop timing
- Result is (**possibly**) more deterministic performance

Harmonic Loop Periods

- Unless inner loop times are enforced, the system timing (outer loop) can be quite variable
- `while()` loop might take
 - Almost no time,
 - L, M, or N ms,
 - L+M or L+N ms,
 - M+N ms, or
 - L+M+N ms
- The results for A, B, or C might happen at any of the possible times, which may be quite undesirable

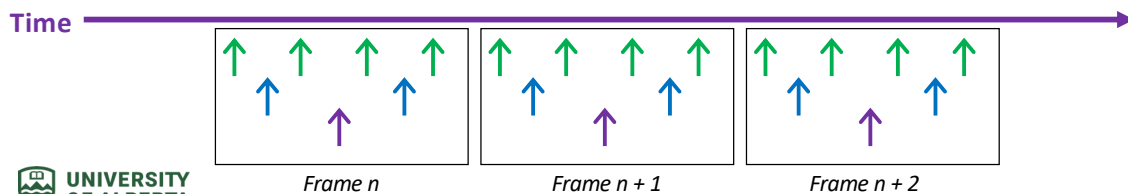
```
static int flag = 0;
while (1) {
    poll_loop_A(); /* might take up to L ms */
    poll_loop_B(); /* might take up to M ms */
    poll_loop_C(); /* might take up to N ms */
} /* end while */
```

Harmonic Loop Periods

- Instead, ensure that each inner loop is harmonic wrt time
- That is, each loop's duration is a multiple of some common time interval
- Ex 1: The loop periods 10 ms, 50 ms & 200 ms are harmonic.
 - Note that $50 = 5 \times 10$, $200 = 20 \times 10$, and $200 = 4 \times 50$.
- Ex 2: The loop periods 10 ms, 25 ms and 133 ms are not harmonic.
 - (Note: 10 ms, 20 ms and 120 ms are harmonic).
- A **single hardware timer** should be responsible for producing the timing of all of the harmonic loop periods.

Why Harmonic Loop Periods?

- Harmonic loop periods can be nested and interleaved so that there is a stable, predictable repeated time **frame structure**
- The total time for the outermost loop within which the events in all shorter loops do not drift in time.
 - They are “forced” to take a fixed amount of time
- Repeated events can be scheduled in the frame so they never drift in time with respect to each other; they will never conflict



Example with Two Non-preemptive Loops

```
void function do_100_ms_loop() { poll_A(); poll_B(); poll_C(); }
void function do_50_ms_loop() { poll_D(); poll_E(); }

initialization_steps();
enum state_type { ST0=0, ST1, ST2, ST3 } state;
static int state = ST0; /* The 100 ms list executes first */
static int flag = 0;
while (1) { /* The frame duration is 100 ms */
    /* wait for ISR of the 40 Hz timer to set flag to 1 */
    while ( !flag ) {} /* wait in idling loop */
    switch ( state ) {
        case ST0: do_100ms_loop();    state = ST1; break;
        case ST1: do_50ms_loop();     state = ST2; break;
        case ST2: /* no loop here */  state = ST3; break;
        case ST3: do_50ms_loop();     state = ST0; break;
    } /* end switch */
    flag = 0; /* cleared for next iteration */
} /* end while */
```

Example with Two Non-preemptive Loops

Here, the inner loops are harmonic with respect to 25 ms

```
void function do_100_ms_loop() { poll_A(); poll_B(); poll_C(); }
void function do_50_ms_loop() { poll_D(); poll_E(); }

initialization_steps();
enum state_type { ST0=0, ST1, ST2, ST3 } state;
static int state = ST0; /* The 100 ms list executes first */
static int flag = 0;
while (1) { /* The frame duration is 100 ms */
    /* wait for ISR of the 40 Hz timer to set flag to 1 */
    while ( !flag ) {} /* wait in idling loop */
    switch ( state ) {
        case ST0: do_100ms_loop();    state = ST1; break;
        case ST1: do_50ms_loop();     state = ST2; break;
        case ST2: /* no loop here */  state = ST3; break;
        case ST3: do_50ms_loop();     state = ST0; break;
    } /* end switch */
    flag = 0; /* cleared for next iteration */
} /* end while */
```



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

23

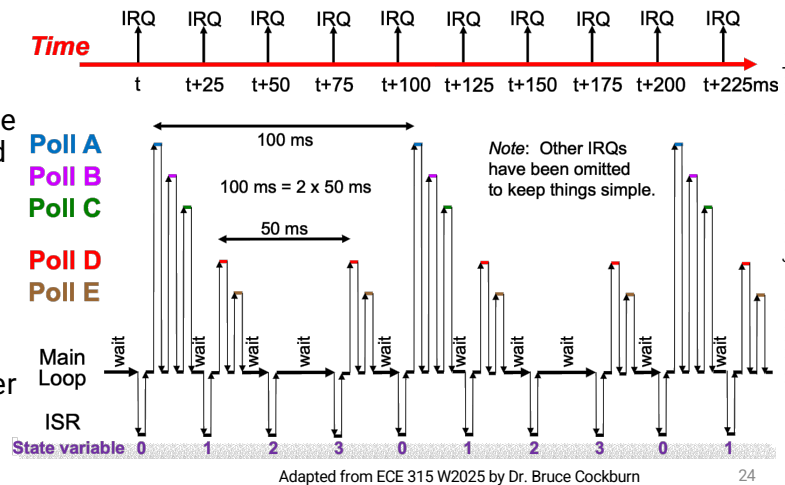
Example with Two Non-preemptive Loops

- Advantages

- Guaranteed min polling frequency
- Guaranteed max response time for polled inputs and devices
- Low latency

- Disadvantages

- Some devices may poll faster than needed
- Software thread had lower cohesion



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

Next Lecture

- More on RTOS
- Tasks and Scheduling
- Protecting Critical Code

Questions

